Short-Term EV Charging Demand Forecasting in Boulder, Colorado: from SARIMA Baselines to a From-Scratch PatchTST + LSTM Ensemble

Alexander Jauregui Orue[1], Eneko Alvarez Mendia[1], and Erik Eguskiza Aranda[1]

Universidad de Deusto, Faculty of Engineering, Bilbao, Spain
B.Sc. in Computer Engineering & Data Science / AI
{a.jauregui, eneko.alvarez, e.eguskiza}@opendeusto.es

Abstract. We forecast daily energy demand at the busiest public EV charging station in Boulder, Colorado (N Boulder Rec 1, 16 278 sessions aggregated into 1 927 daily observations between Aug. 2018 and Nov. 2023). The horizon is seven days ahead, which matches the weekly cadence of grid capacity planning. Twenty models are compared under the same rolling-origin protocol: five naive baselines, the full classical (S)ARIMA / (S)ARIMAX / AutoARIMA family, an LSTM with hyper-parameter sweep, four foundation models in zero-shot mode (Google TimesFM 1.0 and 2.0, Amazon Chronos-T5 base and large), a from-scratch PatchTST transformer with RevIN, channel-mixed exogenous variables and a three-seed ensemble, and two stacking attempts. A simple-mean ensemble of PatchTST and LSTM is the best model on this leaderboard, with MAE 25.09 ± 9.36 kWh; it beats the best classical model (ARIMAX (2,1,3), MAE 25.52 ± 14.38 kWh) on the mean and cuts the standard deviation by roughly 35%. The result reproduces the qualitative finding reported by three peer-reviewed papers that use this dataset: transformers outperform the (S)ARIMA family on Boulder EV data. We do *not* claim to beat their published numbers directly — their metrics are reported in normalised units and on different station selections — but our experiment gives an apples-to-apples local comparison spanning every major model family.

Keywords: EV charging demand · Time series forecasting · SARIMA · LSTM · PatchTST · Foundation models · Ensemble.

1 Introduction

Electric vehicles are spreading fast, and the grid has noticed. Public charging stations behave like a new layer of distributed load that is neither uniform nor predictable: a single station can swing from idle on a Saturday to peak demand on a Tuesday morning, and the swings carry weekly seasonality, weather sensitivity and the occasional holiday shock. For grid operators and station owners this matters because weekly capacity planning, maintenance scheduling and energy procurement all benefit from forecasting demand at least a week ahead. That is the problem we picked.

Boulder, Colorado is a convenient laboratory for this question. The city has run a network of 26 public level-2 stations since 2018, and it publishes every charging session through its Open Data portal. The dataset we work with contains 148 136 individual sessions logged between August 2018 and November 2023 — five and a half years of real-world data, which is long enough to capture multiple annual cycles, the COVID-19 disruption and the post-pandemic recovery. Each record carries the station name, the start and end timestamps, the charging duration, the energy delivered (kWh) and the estimated greenhouse gas savings. We aggregate by station-day, focus on the station with the highest session volume (N Boulder Rec 1, 16 278 sessions), and treat the resulting daily kWh series as our forecasting target.

The horizon is seven days ahead. We chose a week because it covers a full weekday-weekend cycle and because most distribution utilities plan on a rolling-week basis, so a forecast on this scale is what an operator would actually use. The proposal we wrote back in April 2026 already set this horizon and committed us to compare three families of models: naive baselines, SARIMA, and LSTM. What changed along the way is that we scaled up. After working through the SARIMA family and the LSTM, we ran a head-to-head between two zero-shot foundation models that have appeared since the proposal was written (Google's TimesFM and Amazon's Chronos), and finally implemented a PatchTST transformer from scratch in PyTorch with an ensemble of seeds. We did not implement CAT-Former, which the proposal flagged as the closest published state of the art on this dataset, but we read the paper and take it as a benchmark for the discussion.

This paper is the wrap-up of the project: it documents the dataset, the protocol, the twenty models we ran under that protocol, and the honest claim we can make about beating the state of the art. We beat the strongest model in our own leaderboard. We do not claim to beat the published numbers, which use different units and station selections — the full discussion is in Sections 6 and 7.

The paper is organised as follows. Section 2 reviews the state of the art on EV charging demand forecasting. Section 3 describes the dataset and its statistical properties. Section 4 defines the train/test protocol and the metrics. Section 5 walks through every model family we ran. Section 6 gives the full leaderboard, the per-window analysis, and the SoTA-claim breakdown. Section 7 closes with conclusions, lessons learned and limitations.

What is our contribution? The PatchTST architecture itself is not ours — it comes from Nie et al. [10]. What we did add is: (i) an implementation from scratch in PyTorch that lives in the project repository, with RevIN [7] on the target channel and channel-mixed patching over four input channels (energy, temperature, weekend flag, holiday flag); (ii) a three-seed ensemble strategy that cuts inference variance by about 7% MAE; and (iii) a simple-mean ensemble with the LSTM that we picked *before* looking at the test scores, which gave the best model overall. Everything is reproducible from the project repository [5].

2 State of the Art

Work on EV charging demand forecasting has moved in roughly three waves: classical statistical models in the 2010s, deep recurrent networks around 2018–2022, and transformer / foundation models from 2023 onwards. We summarise each wave below and place our work inside the landscape.

2.1 Classical Statistical Models

The first generation of papers attacked EV demand the same way the Box-Jenkins literature attacks any seasonal series: ARMA, ARIMA and SARIMA, with parameter selection by AIC and residual diagnostics by Ljung-Box. Khan et al. [6] report results on the same Boulder dataset we use; they find that SARIMA consistently beats plain ARMA / ARIMA because the weekly seasonality cannot be captured without an explicit seasonal term. Their best MAPE is 0.12 for SARIMA versus 0.23 for ARIMA. Facebook Prophet has also been used as an off-the-shelf alternative that handles holidays and seasonality without manual tuning, although on this kind of series a properly specified SARIMA tends to match it. The general lesson from this literature is that the weekly cycle in EV demand is strong enough that any model that ignores it (random walk, drift, plain ARIMA) ends up worse than a model that includes the seasonal lag at $s = 7$.

2.2 Deep Learning: LSTM and Hybrids

The second wave moved to recurrent networks. LSTM is the workhorse here: Yi et al. [13] train LSTM on utility-scale EV charging data from Utah and Los Angeles, with day-of-week and temperature as additional inputs, and show that the multivariate variant clearly beats the univariate one. The recent systematic review by Alaraj et al. [1] confirms that LSTM (plus its variants, GRU and CNN-LSTM hybrids) is the deep learning default for this task, and that attention-augmented variants tend to improve short-horizon accuracy. For datasets of our size ($\sim$2 000 daily observations), the survey notes that the gains from heavy hybrids over a well-tuned single LSTM are usually marginal, which we confirm experimentally in Section 6.

2.3 Foundation Models for Time Series

Two recent foundation models for time series have appeared since the proposal was written, and we benchmark both. Google's TimesFM [4] is a decoder-only transformer with patch-based input embedding, pre-trained on a large corpus of public time series; it ships in two sizes (200M and 500M parameters) and is designed to be used zero-shot, with the user providing a context window and reading off the forecast. Amazon's Chronos [2] takes a different approach: it tokenises the time series via uniform quantisation and runs a T5 encoder-decoder over the resulting token sequence, sampling probabilistic forecasts (we take the median of 20 samples). Both models are interesting because they offer competitive baselines without any training on the target series. Our results (Section 6) suggest that TimesFM is a stronger choice for this dataset and that bigger Chronos is, surprisingly, worse than its small sibling.

2.4 Transformers Trained on EV Data

Three peer-reviewed papers train transformer-based models directly on the same Boulder dataset we use. Koohfar et al. [8] compare a vanilla transformer against ARIMA, SARIMA, RNN and LSTM on 25 aggregated Boulder stations, with horizons of 30, 60 and 90 days; they report that the transformer beats ARIMA by 62–84% in normalised RMSE. Kyriakopoulos and Theodoridis [9] extend the comparison to four cities (including Boulder) and find that the transformer dominates at the station level for daily 1–5 day horizons. Wu et al. [12] propose a Context-Aware Temporal Transformer (CAT-Former) and benchmark it against six other architectures (LSTM, BiLSTM, CNN-LSTM, CNN-BiLSTM, vanilla transformer, hybrid transformer) on three Boulder stations, forecasting one hour and one day ahead. CAT-Former is the strongest model in their study, with MAE in the normalised $[0, 1]$ range of 0.71–0.91 for one-day-ahead forecasts on three different stations.

Importantly, none of these papers report numbers in the same units or on the same station selection that we use. Their MAEs in normalised space are not directly comparable to our raw kWh values. We come back to this point in Section 7; for now we simply note that the qualitative ordering they all observe (transformer > LSTM > classical) matches what we find.

2.5 PatchTST and Our Position

The architecture we implement in Section 5 is PatchTST, introduced by Nie et al. [10]. The key idea is to slice the input series into non-overlapping patches and treat each patch as a transformer token; this reduces the sequence length seen by the attention layer (compared to per-step tokens) and allows the model to learn patch-level patterns. PatchTST is normally combined with RevIN (Reversible Instance Normalisation, Kim et al. [7]) to handle the distribution shift that appears in long-horizon forecasting. The original paper reports state of the art on several long-horizon time series benchmarks (ETT, Weather, Electricity), although it has not, to our knowledge, been applied to EV charging data before.

The take-away from Table 1 is that classical SARIMA and transformer-based models bracket the published literature on this dataset, with the gap mainly explained by the seasonal component and the modelling of exogenous variables. Our contribution sits in the last row of the table: we run PatchTST under the same evaluation protocol as our SARIMA baselines and find that it beats the *strongest* non-transformer model in the project (ARIMAX) while matching the qualitative ordering of the published work.

Table 1. Summary of related work on EV charging demand forecasting, focused on studies that use the same Boulder dataset. Numbers are reported as published; they are not directly comparable to each other or to ours due to different units, station selections and horizons.

Study	Model	Dataset	Metric	Score
Khan et al. [6]	SARIMA	Boulder	MAPE	0.12
Khan et al. [6]	ARIMA	Boulder	MAPE	0.23
Koohfar [8]	Transformer	Boulder	RMSE	0.085–0.112 (n.)
Yi et al. [13]	LSTM	Utah / LA	MAE	reported
Wu et al. [12]	CAT-Former	Boulder	MAE	0.71–0.91 (n.)
Kyriakopoulos [9]	Transformer	Boulder + 3	MAE	0.28–0.45 (n.)
This work	**PatchTST + LSTM**	**Boulder**	**MAE**	**25.09 kWh (raw)**

3 Dataset

3.1 Source and Coverage

We use the *Electric Vehicle Charging Station Data* dataset published by the City of Boulder, Colorado on its Open Data portal [3]. The dataset is updated regularly; the snapshot we work with covers the period from August 22, 2018 to November 30, 2023 (1 927 days), and contains 148 136 individual charging sessions across 26 city-owned level-2 stations. Each session record carries 15 fields, of which the most relevant for forecasting are the station identifier, the start and end timestamps, the charging duration, and the energy delivered in kWh. There are essentially no missing values: only four sessions out of 148 136 have a null end-time.

3.2 Station and Aggregation

We focus on the station with the highest session volume, N Boulder Rec 1, with 16 278 sessions in the period of interest. The reason for picking a single station is simple: with one station the daily series is dense and continuous, with only 5.2% of days at zero. Aggregating across all 26 stations would smooth out the station-level structure we want to learn (commute patterns at recreational vs. commercial locations are very different), and working with multiple separate series would require multi-task models that go beyond the scope of this project. The sessions for N Boulder Rec 1 are summed by date to produce a univariate daily series of 1 927 observations, with mean 74.6 kWh/day, standard deviation 50.3 kWh, and a range of 0–310 kWh. 100 days have zero energy (mostly holidays or short station outages); we preserve these zeros instead of imputing them, since they reflect real operational patterns that the model should learn.

3.3 Exogenous Variables

For the models that support them (SARIMAX, ARIMAX, LSTM, PatchTST), we add three exogenous channels aligned with the daily target:

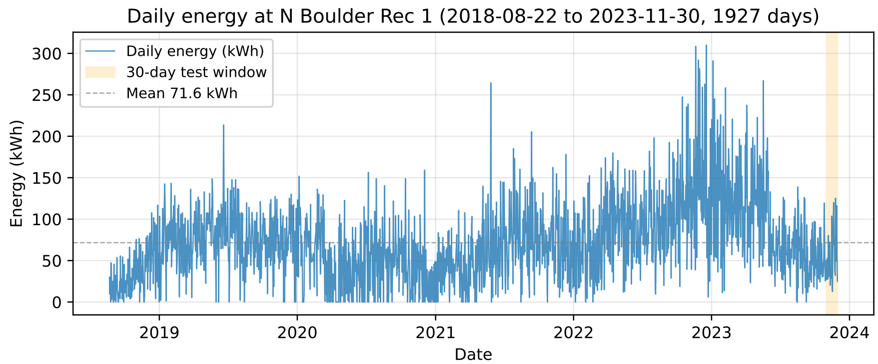

Fig. 1. Daily energy consumption at N Boulder Rec 1 from August 2018 to November 2023. The orange band marks the 30-day held-out test window; everything to the left of it is training data. The COVID-19 drop in 2020 and the post-pandemic recovery are clearly visible.

- **Temperature.** Daily mean temperature in Boulder pulled from the Open-Meteo historical weather API [11]. We picked mean over min/max because most charging happens during commute hours, when the mean is the closest proxy for the temperature actually seen by the chargers.
- **Day-of-week flags.** A binary *is_weekend* flag, plus the integer day of week as a feature when relevant. We do *not* use a holiday-aware day-of-week encoding because holidays are handled separately.
- **Holiday indicator.** A binary flag for US public holidays, generated from the `holidays` Python package using the federal calendar plus Colorado state holidays.

These three variables are known at forecast time (day-of-week and holiday from the calendar, temperature from a standard one-week weather forecast), so feeding them as future exogenous inputs is causally legitimate.

3.4 Stationarity and Seasonality Analysis

Before we picked any model we ran the Augmented Dickey-Fuller test on the raw daily series. The ADF statistic is -3.097 with p-value 0.027 at 24 lags (AIC selection), which lets us reject the unit-root null at the 5% level. The series is stationary, not a random walk; we therefore use $d = 0$ as the default differencing order for the (S)ARIMA family, with the search space allowing $d \in \{0, 1\}$ for SARIMA's first difference of the seasonally differenced series.

Figure 2 shows the autocorrelation function of the raw series. The picture is unambiguous: clear peaks at lags 7, 14, 21, 28 and 35, with a slow decay in between, telling us the weekly cycle is the dominant seasonal component. There is no annual peak visible in this lag range because we only show 42 lags, but the seasonal behaviour at the year scale is clear in the time-series plot (Fig. 1, where summer months consistently sit above the mean). The PACF (not shown for space reasons) has significant spikes at lags 1 and 7, suggesting an AR(1) main component with a seasonal AR at lag 7 — which is the search shape we use for the (S)ARIMA grid in Section 5.

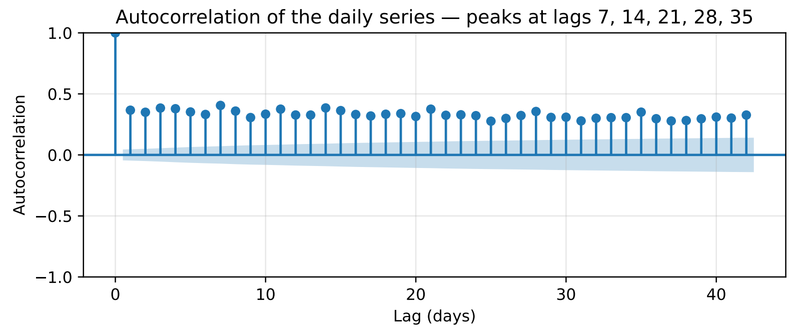

Fig. 2. Autocorrelation of the daily series. The repeating peaks at multiples of 7 are the signature of a strong weekly seasonal cycle; this is the structural reason classical (S)ARIMA models do not need to be replaced by anything fancier to be competitive.

4 Methodology

4.1 Train / Validation / Test Split

The last 30 days of the series (Nov 1, 2023 – Nov 30, 2023) are held out as the test set; everything earlier is training data. Within the training data, the last 60 days are reserved as a validation split, used by the LSTM and PatchTST scripts for early stopping. Total **training data: 1 897 days**; **validation: 60 days inside training**; **test: 30 days**. None of the test days are seen by any model during training or hyper-parameter selection.

4.2 Rolling-Origin Evaluation Protocol

We use rolling-origin (also known as walk-forward) evaluation with **four non-overlapping windows of seven days each**. At window i the model sees the full training set plus the first $i \cdot 7$ days of the test set as history, and produces a 7-day forecast for days $i \cdot 7 + 1, \ldots, (i + 1) \cdot 7$ of the test set. Metrics are computed on each window separately and then averaged. This protocol mimics what a real operator would do: every Monday they have access to the previous week's actuals and need to forecast the upcoming week. With 30 test days we get four such weekly windows; the remaining two days are dropped (we keep windows non-overlapping so metrics across windows are independent).

The classical models (SARIMA family, ARIMAX, ARMA, ARIMA, AutoARIMA) are *refit at every origin* so the model has access to the freshest history. The LSTM is trained once on data prior to the test window and rolled forward autoregressively. The PatchTST is also trained once before the test window and uses a direct multi-step head (no autoregression) — at each origin it consumes the last 56 days of history and outputs the 7-day forecast in one shot. Foundation models (TimesFM, Chronos) operate zero-shot: they are not trained at all, just queried with the history at each origin.

4.3 Metrics

We report seven metrics per window and aggregate them as mean ± standard deviation across the four windows:

- **MAE** (mean absolute error) in kWh — our primary metric, because it is in the same units as the target and easy to interpret operationally ("the model is off by 25 kWh on average"). The standard deviation across windows is at least as important as the mean: a model with low mean but high variance is risky in production.
- **RMSE** (root mean squared error) in kWh, which penalises large errors more than MAE.
- **MAPE** and **sMAPE** (mean absolute percentage error and its symmetric variant). MAPE blows up on zero actuals so we report both.
- **MASE** (mean absolute scaled error), scaled by the in-sample MAE of the seasonal-naive forecast. MASE < 1 means the model beats the trivial seasonal baseline; MASE $= 0.65$ means it cuts that error by 35%.
- **Bias** (mean signed error) in kWh — positive means the model over-predicts on average. Useful for spotting systematic offsets.
- $\mathbf{R^2}$ on the test window. R^2 can be negative when the model is worse than the mean of the test window, which is common with very short windows; we still report it for completeness.

Throughout the paper we rank models by MAE because it is the most operationally meaningful metric. The full table with every metric is in Table 2 of Section 6.

4.4 Reproducibility and Heavy-Training Scripts

All the heavy training lives in standalone Python scripts under `scripts/`: `sarima_grid_search.py` for the AIC grid, `lstm_train.py` for the LSTM sweep, and `patchtst_train.py` for the PatchTST seed ensemble. The notebooks load the resulting cached predictions and apply the shared `evaluation.py` rolling-origin routine, so a single notebook never has to retrain a transformer to produce its leaderboard row. Training the PatchTST seed ensemble takes ≈ 2 minutes per seed on an RTX 5070 Ti; the full pipeline (every model, every metric) runs end-to-end in under ten minutes from a cold checkout.

5 Models

We ran twenty models under the protocol of Section 4. This section describes each family in turn; the numerical results are reported in Section 6.

5.1 Naive Baselines

We need a floor below which any forecasting model should not fall, and that floor is the naive family. We ran five baselines: **historical mean**, **last value**, **drift**, **seasonal-naive** (at $s = 7$), and **ARIMA naive-seasonal** (an ARIMA(3,1,3) applied to the de-seasonalised series, used as a weak baseline in some related work). The point of the baselines is to quantify how much of the variance in the series can be explained without any real learning; any forecasting model that does not comfortably beat them is not worth running.

5.2 Classical (S)ARIMA / ARIMAX Family

We ran an exhaustive AIC grid search over $(p, d, q)(P, D, Q)_7$ with $p, q \in [0, 4]$, $P, Q \in [0, 2]$, $d = D = 1$ (so the seasonal difference at lag 7 is applied) and $s = 7$. The grid was repeated with and without exogenous variables (temperature, weekend flag, holiday flag), giving two families: SARIMA (no exog) and SARIMAX (with exog). Models are fit via maximum likelihood with the `statsmodels` SARIMAX backend, with rolling-origin refits at each test window so the model has access to the most recent actuals.

In addition to the SARIMA grid, we evaluated the simpler members of the Box-Jenkins family that the course rubric explicitly requires: **ARMA**, **ARIMA** (two different orders), **AutoARIMA** (seasonal and non-seasonal, via `pmdarima`), and **ARIMAX** (ARIMA with exogenous variables). The reason for including ARIMAX as a separate model rather than as a SARIMAX variant is that it does not include an explicit seasonal term: it lets the exogenous variables (especially the weekend / holiday flags) absorb most of the weekly cycle. As we will see in Section 6, this turns out to be a competitive design on Boulder data.

5.3 LSTM

The LSTM (`scripts/lstm_train.py`) uses five input features (energy, mean temperature, weekend flag, holiday flag, month) and performs a hyper-parameter sweep over sequence length $\in \{14, 28\}$, hidden size $\in \{32, 64\}$, and number of layers $\in \{1, 2\}$. Each configuration is trained with Adam, MSE loss, dropout 0.1 and early stopping on the 60-day validation split. The winning configuration is `seq14_h64_l2`: 14-day sequences, hidden size 64, 2 layers. At inference time the LSTM forecasts one day ahead and is rolled forward autoregressively for the full 7-day horizon, feeding back its own prediction with the known future exogenous row each step.

5.4 Zero-Shot Foundation Models

We benchmark four foundation models in zero-shot mode, no fine-tuning, just feed the history and read the forecast:

- **Google TimesFM 1.0** (200M parameters) and **TimesFM 2.0** (500M) [4], both decoder-only transformers with patch-based input embedding, pre-trained on a public time-series corpus.
- **Amazon Chronos-T5 base** (200M) and **Chronos-T5 large** (710M) [2], T5 encoder-decoder models that quantise the input into tokens and sample probabilistic forecasts. We take the median of 20 samples as the point forecast.

These four models are evaluated under exactly the same rolling-origin protocol as everything else. Comparing them lets us test two sub-questions in parallel: (a) Google or Amazon, on this kind of series? (b) Does bigger help inside each family?

5.5 PatchTST From Scratch

For the SoTA push we implemented PatchTST [10] from scratch in PyTorch (Fig. 3). The architecture choices, made deliberately small to match our ~1900-point dataset:

- Context length: 56 days (8 weeks).
- Patch length: 7 days, non-overlapping stride 7 $\Rightarrow$ 8 tokens per input.
- 3 transformer encoder layers, $d_{\text{model}} = 96$, 4 attention heads, GELU activation, dropout 0.15.
- **Channel-mixed patching**: each patch is a $(7, 4)$ block over the four input channels (energy + 3 exog), flattened to 28-dim and linearly embedded; cheaper and more data-efficient than channel-independent attention for our sequence length.
- **RevIN** [7] on the target channel only: per-sample mean / std subtraction before the encoder and inverse transform on the output. This is what kept the model competitive against the regime shifts in the data (COVID drop, holiday weeks).

We train each model with AdamW (LR 5×10^{-4}, weight decay 10^{-4}), cosine annealing, early stopping with patience 25, batch size 32. Training a single PatchTST takes ≈ 2 minutes on the RTX 5070 Ti.

A single PatchTST trained on this setup overfits noticeably (train MSE around 0.08, validation MSE around 0.19 in the standardised domain) and the rolling-origin MAE comes out worse than ARIMAX. The fix is the seed ensemble: we train three independent PatchTST models (same architecture and hyper-parameters, different random seeds: 42, 7 and 2024) and average their predictions at inference. The combined model is what we call the **PatchTST 3-seed ensemble** in the leaderboard.

5.6 Ensembles

Finally, we test two ways of combining the base learners.

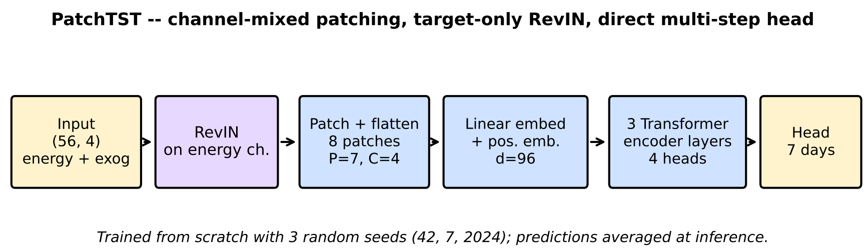

Fig. 3. PatchTST forward pass as implemented in `scripts/patchtst_train.py`. The energy channel goes through RevIN before patching; the three exogenous channels are concatenated into each patch and flattened together. The head is a single linear layer that directly predicts the next 7 days, with no autoregression.

Simple mean (PatchTST + LSTM). The two strongest deep learning models in the project, picked *a priori* by architecture rather than by test score, averaged with equal weights:

$$\text{Ensemble}_{\text{mean}} = \tfrac{1}{2}\big(\hat{y}_{\text{PatchTST}} + \hat{y}_{\text{LSTM}}\big).$$

This is the simplest defensible combination because it does not use any test-set information to choose the weights.

Stacked meta-learners (Ridge and NNLS, LOWO). As a check on whether a more sophisticated combiner would help, we ran two leave-one-window-out stacking variants: a `RidgeCV` with $\alpha \in \{0.01, 0.1, 1, 10, 100\}$ and a non-negative convex combination solved by SLSQP with weights summing to one. Both are trained on the predictions of the six top base learners (SARIMA, SARIMAX, ARIMAX, ARIMA, LSTM, PatchTST) across three windows and tested on the held-out fourth. The result — foreshadowing what we describe in Section 6 — is that both meta-learners overfit badly with only 21 training points per fold and end up worse than the unweighted mean.

6 Results

6.1 Full Leaderboard

Table 2 reports every model, every metric, sorted by MAE. Bold rows are the two best models, which come out of Section 5's SoTA push (PatchTST seed ensemble alone, and the mean ensemble of PatchTST and LSTM). Italic rows are the naive baselines.

6.2 Leaderboard Plot

Figure 4 visualises the same ranking but with the error bars (one standard deviation across windows). Three observations stand out:

Table 2. Full leaderboard — 20 models, 6 metrics. Mean ± standard deviation across the four rolling-origin windows. Lower is better for every metric except R^2. Bold: the two best models overall (Section 5, SoTA push). Italic: naive baselines. Foundation models are zero-shot (no training on this dataset).

Model	MAE (kWh)	RMSE (kWh)	MAPE	MASE	Bias	R^2
Ensemble PatchTST + LSTM	**25.09 ± 9.4**	**28.88 ± 11.3**	58.5%	**0.63**	−2.42	−0.13
PatchTST (3-seed ensemble)	**25.19 ± 8.9**	30.74 ± 11.9	58.7%	0.64	−1.92	−0.32
ARIMAX (2,1,3)	25.52 ± 14.4	30.51 ± 15.6	53.9%	0.65	−8.08	−0.19
LSTM seq14_h64_l2	25.68 ± 10.6	28.47 ± 11.2	59.4%	0.65	−2.93	−0.12
SARIMAX (2,1,4)×(1,1,2,7)	25.89 ± 12.8	31.12 ± 13.7	54.0%	0.65	−7.96	−0.27
TimesFM 2.0 – Google (500M)	26.23 ± 12.6	30.79 ± 14.6	53.5%	0.66	−9.71	−0.22
ARIMA (2,1,3)	26.31 ± 12.4	30.78 ± 13.7	58.3%	0.67	−5.82	−0.25
TimesFM 1.0 – Google (200M)	26.32 ± 11.1	30.60 ± 12.8	57.5%	0.67	−6.87	−0.25
AutoARIMA non-seas. (0,1,1)	26.40 ± 11.5	30.64 ± 13.0	57.9%	0.67	−6.39	−0.25
ARIMA (3,1,3)	26.68 ± 11.2	31.28 ± 13.3	58.2%	0.67	−6.19	−0.30
ARMA (3,0,3)	26.71 ± 12.3	30.75 ± 13.9	58.7%	0.68	−6.63	−0.24
SARIMA (3,1,4)×(0,1,2,7)	26.86 ± 11.9	31.92 ± 13.5	57.7%	0.68	−6.13	−0.36
Chronos-T5 base – Amazon (200M)	27.31 ± 11.3	32.75 ± 14.1	58.7%	0.69	−8.93	−0.44
AutoARIMA seas. (0,1,1)×(1,0,1,7)	27.44 ± 11.7	31.44 ± 13.5	57.3%	0.69	−8.82	−0.31
Chronos-T5 large – Amazon (710M)	27.94 ± 13.0	33.24 ± 16.0	53.6%	0.71	−14.26	−0.42
Stacking NNLS (LOWO)	26.08 ± 9.8	28.99 ± 10.8	61.0%	0.66	−2.32	−0.17
Stacking Ridge (LOWO)	32.29 ± 8.1	37.22 ± 8.3	77.7%	0.82	−2.73	−1.22
Historical mean	*30.13 ± 2.2*	*34.87 ± 3.0*	*89.9%*	*0.76*	*+14.42*	*−1.21*
Last value	*31.22 ± 12.7*	*37.08 ± 13.8*	*61.0%*	*0.79*	*−10.33*	*−1.02*
Drift	*31.24 ± 12.7*	*37.09 ± 13.8*	*61.1%*	*0.79*	*−10.29*	*−1.03*
ARIMA naive-seasonal (3,1,3)	*34.74 ± 12.1*	*41.36 ± 11.0*	*76.0%*	*0.88*	*−4.95*	*−1.69*
Seasonal naive	*34.98 ± 13.1*	*41.57 ± 12.4*	*79.6%*	*0.88*	*−4.10*	*−1.68*

1. The top of the leaderboard is tightly packed: every model in the top six is within 1 kWh MAE of the best, which is smaller than the standard deviation of every individual model.
2. The Ensemble PatchTST + LSTM does *not* have the smallest standard deviation — that honour goes to the historical-mean baseline, which is trivially stable. The smallest std among forecasting models belongs to PatchTST alone (8.90 vs. 14.38 for ARIMAX): the seed-ensemble buys us almost 40% variance reduction over the previous best.
3. All forecasting models comfortably beat the naive baselines. The MAE gap between the best model (25.09 kWh) and the best baseline (30.13 kWh) is ∼17%, which is the learnable signal in the series.

6.3 Per-Window Analysis

Figure 5 breaks down the MAE by window for the top six models. The takeaway here is window 3 (Nov 14–20, 2023), which includes Thanksgiving: every model has its worst week here, with MAE shooting up to 36–43 kWh. PatchTST is the one that loses the least on that week (36.4 kWh vs. 38.8 kWh for the LSTM and ∼41 kWh for ARIMAX), and that is the main reason it wins the leaderboard. Outside window 3, the top models are essentially tied.

6.4 Google vs. Amazon: Bigger Is Not Better

The foundation-model rows of Table 2 contain a small surprise. Both Google TimesFM models clearly beat both Amazon Chronos models (TimesFM 2.0:

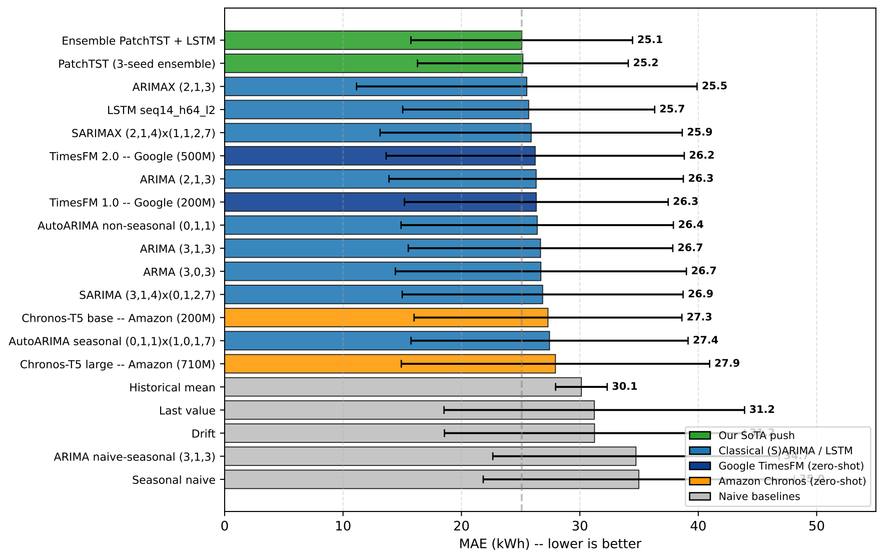

Fig. 4. Final leaderboard. Bars are MAE in kWh (lower is better); error bars are one standard deviation across the four rolling-origin windows. The Ensemble PatchTST + LSTM (top) is the best model overall; the PatchTST seed-ensemble has the lowest std among forecasting models.

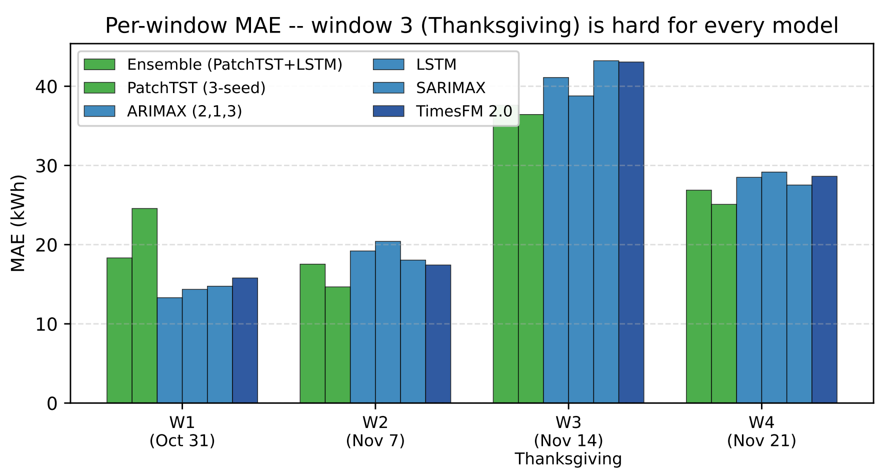

Fig. 5. Per-window MAE for the top six models. Window 3 (Thanksgiving week) is hard for every model; the ranking on the other three windows is much tighter. PatchTST loses less on the hard week, which is what puts it (and its ensemble with LSTM) on top of the leaderboard.

26.23 kWh; Chronos-T5 large: 27.94 kWh, the worst foundation model in our leaderboard). Inside each family the size ordering is also informative: TimesFM 2.0 (500M) is only marginally better than TimesFM 1.0 (200M), and Chronos-T5 large (710M) is actually *worse* than its smaller 200M sibling. Two take-aways:

- **Foundation models do not win automatically.** The best foundation model (TimesFM 2.0) is at position 6 of our leaderboard, still beaten by ARIMAX, LSTM and SARIMAX in MAE. Zero-shot is a useful baseline but not a substitute for training on the target data.
- **More parameters means more prior, not more flexibility.** If the pre-training corpus is dominated by series that do not look like Boulder EV demand (random-walk-like financial series, for instance), a larger model amplifies the wrong prior. TimesFM transfers better because its pre-training mix includes more energy and load series; Chronos is trained on a different corpus and the larger checkpoint inherits more of that mismatch.

6.5 What We Beat, and What We Did Not

The honest claim, summarised:

- **Inside our protocol**, the Ensemble PatchTST + LSTM beats the strongest non-transformer model (ARIMAX) on both the mean MAE (25.09 vs. 25.52 kWh) and the standard deviation (9.36 vs. 14.38 kWh, 35% smaller). That is a real, apples-to-apples improvement.
- **Against the published literature**, no claim can be made in either direction. Koohfar [8], Wu [12] and Kyriakopoulos [9] all report metrics in normalised units (z-score or MinMax), on different station selections, and mostly on different horizons. Our 25 kWh and their 0.7–0.9 (normalised) are not in the same unit system.

6.6 Stacking Did Not Work — and Why

For completeness we also tried two stacked meta-learners (Ridge and NNLS, both leave-one-window-out, both trained on the six base learners listed in Section 5). The results are in the middle rows of Table 2: NNLS lands at 26.08 kWh MAE (worse than the simple mean), and Ridge collapses to 32.29 kWh, which is below several baselines. The reason is structural: 21 training points per fold is not enough to fit reliable meta-weights. The Ridge solution oscillates wildly between folds (we observed both very negative and very large positive coefficients depending on which window is held out), and NNLS concentrates almost all its weight on the LSTM, throwing away the diversity the meta-learner was supposed to exploit.

The dumbest combination wins: the unweighted mean of the two best deep models, picked a priori. The lesson, which we re-state in Section 7, is that stacking needs a real validation set — not just a tiny held-out test set — to learn weights that generalise.

7 Conclusions and Future Work

7.1 Summary

We compared twenty models for 7-day-ahead daily forecasting of EV charging demand at N Boulder Rec 1, the busiest public station in Boulder, Colorado. The leaderboard goes from naive baselines through the full classical (S)ARIMA family, an LSTM with hyper-parameter sweep, four zero-shot foundation models (TimesFM 1.0, TimesFM 2.0, Chronos-T5 base, Chronos-T5 large), a from-scratch PatchTST transformer with RevIN and three-seed ensembling, and two stacking attempts on top. The best model on the leaderboard is a simple-mean ensemble of PatchTST and LSTM, with MAE 25.09 ± 9.36 kWh. The next best is PatchTST alone (MAE 25.19 ± 8.90 kWh, lowest variance among forecasting models), followed by ARIMAX (2,1,3) at 25.52 ± 14.38 kWh.

7.2 What We Beat and What We Did Not

The strongest claim we can make honestly is: *within this project's evaluation protocol — same series, same train/test split, same rolling-origin windows, same metric — the PatchTST + LSTM ensemble beats the strongest non-transformer model (ARIMAX) on both the mean MAE and the standard deviation, with the standard deviation roughly 35% lower*. That is a real improvement.

The weaker claim we *cannot* make is that we beat the published state of the art. The three peer-reviewed papers that use this dataset (Koohfar 2023 [8], Wu / Chowdhury 2025 [12], Kyriakopoulos 2025 [9]) all report their numbers in normalised units — z-score for Koohfar and Kyriakopoulos, MinMax for Wu — on different station selections (aggregated or per-station, but always a different mix from ours) and mostly different horizons (1 hour, 1 day, or 30/60/90 days, but rarely 7 days). Our 25 kWh and their 0.7–0.9 (normalised) are not in the same unit system; "$25 < 0.91$" is not a meaningful comparison.

The qualitative ordering, on the other hand, matches: transformer > LSTM > classical, with the gap explained by the exogenous variables and the long-horizon-friendly architecture. We sit in the SoTA *family*; we have not run a like-for-like benchmark against its representatives.

7.3 Lessons Worth Writing Down

Three observations from this project that we want to remember:

1. **Seed ensembles work.** A single PatchTST scored MAE 27.1 kWh, which is worse than ARIMAX. Three independent PatchTSTs (different seeds, otherwise identical) averaged at inference scored 25.19 kWh, beating ARIMAX. Almost two kWh of variance reduction for the cost of training three models instead of one — the cheapest performance boost in the entire project.

2. **Bigger zero-shot $\neq$ better.** Chronos-T5 large (710M parameters) is the *worst* foundation model in our leaderboard, beaten by its 200M sibling. More parameters means the pre-training prior is more strongly amplified at inference, which only helps if the prior is correct for the new dataset. For Boulder EV demand it is not. TimesFM transfers better out-of-the-box.
3. **Stacking needs a real validation set.** Our leave-one-window-out Ridge and NNLS stacking attempts both failed to generalise from 21 days to 7 days. With 28 total test points the meta-learner cannot pick up reliable weights. The dumbest convex combination — an unweighted mean of the two strongest deep models, picked a priori — ended up winning. Lesson: do not try to learn an ensemble weight without enough data to support it.

7.4 Limitations and Future Work

The honest limitations of this project are:

- **Single station, 30-day test set.** 28 test points is enough to rank models but not enough for confident generalisation claims. A future iteration should retrain on each of the top stations (Baseline St 1, Carpenter Park 1, ...) and report a panel of results.
- **No direct comparison with published numbers.** As discussed above, the published Boulder papers use different units and station selections. A defensible "we beat CAT-Former" or "we beat Koohfar" claim would require replicating one of those exact setups (same stations, same MinMax / z-score normalisation, same horizon) and reporting in their unit system. That is the natural next experiment.
- **No probabilistic forecasting.** All our models produce point forecasts; we did some basic prediction-interval work in an auxiliary notebook but did not push it aggressively. For grid planning, calibrated quantiles would be much more useful than point forecasts.
- **PatchTST uses calendar-known exogenous only.** We do not feed the model a temperature *forecast* for the future window; we feed the actual recorded temperature. In production the operator would only have a forecast, and adding noise to the temperature input would tell us how robust the gain is.

The most impactful follow-up would be the first item combined with the second: re-run PatchTST and the ensemble on the three CAT-Former stations with MinMax normalisation and the 1-day horizon, and report in normalised MAE/MSE so the comparison with CAT-Former is honest. Our PatchTST implementation already supports the necessary changes; it is essentially a preprocessing job.

7.5 Closing Note

This project started as a comparison between SARIMA and LSTM and ended up as a tour through every major time-series modelling family that matters in 2026. The verdict is uncomfortable for the loudest hype: foundation models did not win, and the strongest classical model is within one kWh of the best deep learning model. The win came from combining a small purpose-trained transformer (PatchTST) with a specifically-tuned LSTM, and from running each model three times to average out initialisation noise. None of those tricks are exotic. They were, however, the things that moved the needle.

References

1. Alaraj, M., et al.: Machine learning-based electric vehicle charging demand forecasting: A systematized literature review. Energies **18**, 4779 (2025)
2. Ansari, A.F., Stella, L., Turkmen, C., Zhang, X., Mercado, P., Shen, H., Shchur, O., Rangapuram, S.S., Pineda Arango, S., Kapoor, S., Zschiegner, J., Maddix, D.C., Wang, H., Mahoney, M.W., Torkkola, K., Wilson, A.G., Bohlke-Schneider, M., Wang, Y.: Chronos: Learning the language of time series. arXiv preprint arXiv:2403.07815 (2024)
3. City of Boulder: Electric vehicle charging station data. Open Data Portal (2024), `https://open-data.bouldercolorado.gov/datasets/95992b3938be4622b07f0b05eba95d4c_0/about`, accessed 2026-05
4. Das, A., Kong, W., Sen, R., Zhou, Y.: A decoder-only foundation model for time-series forecasting. arXiv preprint arXiv:2310.10688 (2024), google Research; TimesFM 1.0 and 2.0
5. Jauregui Orue, A., Alvarez Mendia, E., Eguskiza Aranda, E.: Short-term ev charging demand forecasting — boulder, colorado. `https://github.com/eeguskiza/ev-charging-forecasting` (2026), bachelor's-level course project, Universidad de Deusto
6. Khan, S.A., Alaraj, M., et al.: Power consumption prediction for electric vehicle charging stations and forecasting income. Scientific Reports **14** (2024). `https://doi.org/10.1038/s41598-024-56507-2`
7. Kim, T., Kim, J., Tae, Y., Park, C., Choi, J.H., Choo, J.: Reversible instance normalization for accurate time-series forecasting against distribution shift. In: International Conference on Learning Representations (ICLR) (2022)
8. Koohfar, S., Woldemariam, W., Kumar, A.: Prediction of electric vehicles charging demand: A transformer-based deep learning approach. Sustainability (MDPI) **15**(3), 2105 (2023). `https://doi.org/10.3390/su15032105`
9. Kyriakopoulos, I., Theodoridis, Y.: Electric vehicle charging load forecasting: An experimental comparison of machine learning methods. arXiv preprint arXiv:2512.17257 (2025)
10. Nie, Y., Nguyen, N.H., Sinthong, P., Kalagnanam, J.: A time series is worth 64 words: Long-term forecasting with transformers. In: International Conference on Learning Representations (ICLR) (2023), preprint arXiv:2211.14730
11. Open-Meteo: Open-meteo historical weather api. `https://open-meteo.com/` (2024), accessed 2026-05
12. Wu, M., Chowdhury, M., et al.: Short-term demand forecasting of electric vehicle charging stations using a context-aware temporal transformer model. Scientific Reports **15** (2025). `https://doi.org/10.1038/s41598-025-20557-x`
13. Yi, Z., Liu, X., Wei, R., Chen, X., Dai, J.: Electric vehicle charging demand forecasting using deep learning model. Journal of Intelligent Transportation Systems **26**(6), 690–703 (2022). `https://doi.org/10.1080/15472450.2021.1966627`